



What Is an Array?

Array = Ordered Memory Boxes

Collection of same type data stored side by side (contiguous), accessed using index. Core building block of DSA.

Simple Example: `arr = [10, 20, 30, 40]`

Index & O(1) Access

Visual:

Index:	0	1	2	3
Value:	10	20	30	40

Key Formula:

$\text{Address} = \text{Base} + (\text{Index} \times \text{Size})$

Explain Simply:

Computer directly jumps to location. No searching. That's why access is **O(1)**.

Memory Visualization (Core Concept)



Table

1000 → 10
1004 → 20
1008 → 30
1012 → 40



Key Idea:

Each integer = 4 bytes
Contiguous memory = fast CPU access



Mention:

This is how arrays work in C

Python List Internals

Code:

```
arr = [10, 20, 30]
```

Visual:

List: [*][*][*]

Heap: 10 20 30

Key Insight:

- List stores pointers
- Not raw values
- Flexible
- Can mix types

Dynamic Resizing (Amortized $O(1)$)

Before:

[10][20][30]

After append:

[10][20][30][40][]

Explain:

- Creates bigger block
- Copies elements
- Deletes old memory

Append = Amortized $O(1)$

📌 This is asked in interviews at: Google, Amazon

Python array Module

Module: array

```
from array import array  
val = array('i', [1,2,3])
```

Key Points:

- Same type only
- More memory efficient
- Closer to real arrays
- Good for understanding DSA fundamentals

NumPy Arrays

Library: NumPy

```
import numpy as np  
arr = np.array([1,2,3])
```

Visual:

[1][2][3]

Why Fast?

- Written in C
- Raw contiguous memory
- Vectorized operations
- CPU cache friendly

Used in AI/ML.

Time Complexity Table

Operation	Complexity
Access	$O(1)$
Update	$O(1)$
Search	$O(n)$
Insert end	$O(1)^*$
Insert middle	$O(n)$
Delete middle	$O(n)$

**Amortized $O(1)$*

 **Highlight:** Shifting makes insertion $O(n)$

Why Contiguous Memory Matters

Contiguous:

[10][20][30][40]

Scattered:

10 20 30

Key Idea:

Better cache usage

Faster CPU access

High performance

This is systems-level understanding.

Beginner → Pro



Stage 1:

- Indexing
- Traversal
- Insert/Delete
- Reverse



Stage 2:

- Two pointers
- Prefix sum
- Sliding window



Stage 3:

- Binary search
- Kadane's algorithm
- In-place optimization



Stage 4:

- Amortized analysis
- Cache optimization
- Memory reasoning